All About The Project With Simplified Explanation:

Let's strip away all the complex tech words. Underneath the jargon, VisionGuard AI is incredibly logical and straightforward. Here is the absolute simplest, plain-English breakdown of your project so that you and your team can understand it perfectly from day one.

1. The Big Picture: What Are We Actually Building?

Imagine a human software tester sitting at a desk. Their job is to open a new website on a phone, a tablet, and a computer. They look for two things:

1. **Visual Bugs: Did the text spill out of a box? Is the menu overlapping the logo?**
2. **Readability (Accessibility): Is the gray text on a white background too hard to read for someone with bad eyesight?**

If they find a mistake, they have to figure out which line of code caused it, rewrite the code, and refresh the page to see if it fixed the problem. This takes hours.

VisionGuard AI is simply a robot that does this exact job in seconds. It opens the website, looks at the screen, finds the ugly or unreadable parts, and hands you the exact code to fix it.

2. How It Works (The 4 Simple Steps)

If someone asks you how the project works, you just walk them through this simple flow:

- **Step 1: The Input. You paste a web link into your dashboard (for example, the IIIT Kottayam student portal) or you upload an image of a web design.**
- **Step 2: The Invisible Browser. Behind the scenes, your app secretly opens a hidden web browser. It takes screenshots of the website in mobile, tablet, and desktop sizes. At the same time, it copies the website's blueprint (the HTML and CSS code).**
- **Step 3: The Brain. The AI looks at the screenshots and the code at the same time. It spots where things look broken (like a button covering up text) and checks if the colors are legally readable.**
- **Step 4: The Fix. The AI writes a small piece of code to fix the bug. But before it shows you, it tests the code secretly in that hidden browser to make sure it actually works. Then, it displays the fixed version on your screen.**

3. What Does the User See? (The Dashboard)

When your professors or users look at your project, they won't see confusing code. They will see a very clean, simple Streamlit dashboard with three things:

1. **The Sliding Picture: A cool interactive slider. On the left, they see the broken website with red boxes highlighting the mistakes. They drag the slider to the right, and they instantly see the fixed, beautiful website.**
2. **The Report Card: A simple list telling them what was wrong (e.g., "The blue button on the mobile screen is too light to read").**
3. **The Code Patch: A small box with the exact lines of CSS code they can copy and paste to fix their website permanently.**

4. Translating the "Fancy Terms" for Your Team

When you are preparing for your viva, the professors will use big words. Here is the cheat sheet to translate those big words into simple concepts for your team:

- **"Multimodal AI"**
    - ***Simple Meaning:*** **Standard AI (like normal ChatGPT) can only read text. *Multimodal* AI has eyes. It can look at a picture (the screenshot) and read text (the code) at the exact same time to figure out a problem.**
- **"Agentic System"**
    - ***Simple Meaning:*** **It works on its own. A normal chatbot waits for you to ask it a question. An *Agent* is given a goal ("Find the bugs") and it clicks the buttons, opens the browser, and tests the code all by itself without you holding its hand.**
- **"Neuro-Symbolic"**
    - ***Simple Meaning:*** **AI is bad at doing exact math. So, we split the brain in two. The "Neuro" part (the AI) looks at the pictures to find messy layouts. The "Symbolic" part is just a strict, old-school Python calculator that does the exact math to check if colors are legally readable.**
- **"Closed-Loop Sandbox"**
    - ***Simple Meaning:*** **A safe test room. We don't trust the AI to write perfect code on the first try. So, it writes the fix, tries it out in a hidden, safe browser (the sandbox), and checks its own homework. If the fix didn't work, it tries again before showing the user.**

5. Breaking Down the Team Workload

Since your 5th semester classes have already started, keeping the work divided cleanly is crucial so nobody gets stressed. Here is the simple version of who does what:

- **The AI Prompter (Role 1): You talk to the AI. You write the instructions that tell the AI exactly how to look at the screenshots and what kind of code to write back.**
- **The Browser Builder (Role 2): You build the invisible browser. You write the Python script that opens the websites and takes the screenshots without crashing your HP Pavilion laptop.**
- **The Rule Checker (Role 3): You are the math person. You write the strict Python rules that read the website's colors and check if they pass standard readability tests.**
- **The Dashboard Designer (Role 4): You build the Streamlit screen. You make the slider, the buttons, and ensure the app looks amazing for presentation day.**

By breaking it down like this, the project stops being a massive, scary enterprise system and becomes four manageable puzzle pieces.

The VisionGuard AI Master Document: 100% Project Knowledge Base:

This is the definitive, comprehensive guide to VisionGuard AI. This document contains every technical specification, workflow, advanced feature, and defense strategy your 4-person team needs to build and present this system for your 5th-semester evaluation at IIIT Kottayam.

1. The Dual-Pipeline Architecture (New Upload Feature Included)

To make this the absolute "best of the best," the system architecture is split into two distinct pipelines. The user can either input a live URL or upload a raw screenshot/wireframe design.

Pipeline A: The Live URL Audit (The Agentic Loop)

1. **Input: User pastes https://example.com.**
2. **Headless Execution: FastAPI triggers Playwright. The browser opens in the background and takes screenshots across Desktop, Tablet, and Mobile viewports.**
3. **DOM Extraction: BeautifulSoup4 scrapes the HTML and CSS.**

4. **Neuro-Symbolic Analysis: The Python math engine calculates WCAG contrast ratios on the DOM colors, while the Multimodal VLM (Gemini/Claude) looks at the screenshot to find overlapping elements.**
5. **Sandbox Repair: The LLM generates a CSS patch, injects it back into Playwright, and takes a "Fixed" screenshot.**
6. **Output: The Streamlit dashboard displays the side-by-side interactive slider and the copy-pasteable CSS code.**

Pipeline B: The Static Screenshot/Figma Upload (The Generative Engine)

1. **Input: User uploads a .png or .jpg of a web design mockup, a Figma wireframe, or a broken UI screenshot.**
2. **Visual Processing: Since there is no underlying DOM code to parse, the system bypasses Playwright and BeautifulSoup.**
3. **Multimodal VLM Heuristics: The Vision model acts as a Senior Frontend UI/UX Reviewer. It analyzes the image for:**
    - **Gestalt Principles: Spacing, alignment, and visual hierarchy.**
    - **Color Theory: Estimating contrast ratios directly from pixel hex codes.**
4. **Code Generation: The VLM reverse-engineers the image and outputs raw, modular frontend code (HTML/Tailwind CSS or React components) required to build or fix the uploaded design.**
5. **Output: The dashboard displays the uploaded image alongside the generated structural code.**

2. The Complete Feature Set (What the App Actually Does)

Your Streamlit dashboard will be a multi-tool for developers. It includes the following elite functionalities:

Feature 1: The Interactive Before/After Slider

Using the streamlit-image-comparison library, users drag a vertical slider across the screen.

- **Left Side: The original website with red bounding boxes drawn via Python's OpenCV around detected visual bugs.**
- **Right Side: The AI-patched version, dynamically re-rendered in the Playwright sandbox.**

Feature 2: WCAG 2.1 Deterministic Math Engine

AI hallucinations are prevented by hardcoding the exact World Wide Web Consortium (WCAG) formulas into Python. The system mathematically proves accessibility compliance. The contrast ratio $C$is calculated from relative luminances L_1(lighter) and L_2(darker):

$$C = \frac{L_1 + 0.05}{L_2 + 0.05}$$

Feature 3: Dynamic Color Blindness Simulator (Elite Addition)

To truly impress evaluators, your Python backend applies color transformation matrices to the captured screenshots to simulate how visually impaired users see the website. For example, to simulate Protanopia (Red-Blindness), the system applies this linear transformation to every pixel's RGB values:

Feature 4: Surgical CSS Patch Export

The dashboard provides a syntax-highlighted code block containing *only* the CSS/HTML needed to fix the bugs. It does not rewrite the whole page, which preserves the user's existing JavaScript and backend routing.

3. The Tech Stack & Local Environment Setup

Deploying this smoothly requires your team to align on the environment, especially when managing asynchronous processes and containerization on local machines.

- **Frontend: Streamlit Community Cloud (for seamless GitHub integration) or a local React/FastAPI combo.**
- **Backend: FastAPI (Python). FastAPI is mandatory because Playwright requires asynchronous execution (async/await) to manage multiple browser viewports simultaneously without blocking the server.**
- **Vision Core: Google Gemini 1.5 Pro (Vision) or Anthropic Claude 3.5 Sonnet.**
- **Environment: Running Playwright and Docker natively on a Windows HP Pavilion x360 requires a properly configured Windows Subsystem for Linux (WSL2) to ensure the headless Chromium instances do not consume all available RAM.**
- **Version Control: GitHub. The repository must cleanly separate the ai_engine, browser_automation, and fastapi_routes into modular directories.**

4. Understanding the Core AI Concepts (For Your Team)

To ensure every team member can defend the architecture, everyone must understand these three principles:

1. **Multimodal VLM (Vision-Language Model): It is not Optical Character Recognition (OCR). The model understands spatial relationships. It knows that a "Submit" button overlapping a "Footer" is a structural error, not just text on a screen.**
2. **Neuro-Symbolic Integration: We do not trust the AI to do math. The Neural engine (AI) finds the subjective layout bugs. The Symbolic engine (Python scripts) does the strict WCAG contrast math.**
3. **Agentic Tool Calling: The LLM is the brain, but it is equipped with hands. It is programmed to output specific JSON commands that trigger Python functions, such as execute_playwright() or draw_bounding_box().**

5. Master Viva Defense Guide: Handling the Bouncers

Your evaluators will try to find holes in your logic. Memorize these defenses.

Bouncer 1: "If the user uploads a screenshot instead of a URL, how do you fix the code if you don't have the original DOM/HTML?"

Defense: "When processing a static image upload, the pipeline shifts from a *Patching* strategy to a *Generative* strategy. Because the underlying DOM is absent, the Multimodal VLM acts as a reverse-engineering engine. It analyzes the visual hierarchy and outputs the boilerplate HTML and Tailwind CSS required to replicate the corrected design. It serves as a rapid prototyping tool rather than a surgical patcher."

Bouncer 2: "Playwright is very resource-heavy. How does your backend handle multiple users submitting URLs at the same time without crashing?"

Defense: "We mitigated this through Operating Systems principles, specifically asynchronous concurrency and caching. FastAPI handles requests asynchronously, meaning I/O bound tasks don't block the main thread. Additionally, we implemented a Redis caching layer. If User B requests an audit for a URL that User A already scanned, the backend fetches the stored JSON report and screenshots from Redis instantly, bypassing Playwright and the VLM entirely."

Bouncer 3: "Why is this better than the Inspect Element tool in Chrome?"

Defense: "Chrome DevTools is a manual diagnostic instrument; it requires a human engineer to actively search for bugs, calculate contrast ratios, and write CSS overrides line-by-line. VisionGuard AI is an autonomous agent. It performs the inspection, mathematical validation, and code writing instantly, across three

different device viewports simultaneously, reducing QA engineering time from hours to seconds."

Bouncer 4: "Are you just passing everything to Gemini and letting it do all the work?"

Defense: "Absolutely not. The LLM is strictly isolated as the reasoning engine for subjective layout anomalies. The heavy lifting of the system—the asynchronous Playwright routing, the DOM Abstract Syntax Tree extraction, the pixel-level color blindness matrix transformations, and the closed-loop sandbox re-rendering—was entirely engineered by our team in Python. The AI is a component of the system, not the system itself."

OUTPUT LOOK:

The output of VisionGuard AI is structured to be practical and developer-friendly. It does not wipe out your existing website to build a completely new one from scratch. Instead, it acts as a surgical Audit, Fix & Preview Engine.

Here is the exact breakdown of how user prompting works and what the final output (OP) looks like in your dashboard.

1. Can the User Prompt and Make Changes to the UI?

Yes. While VisionGuard AI runs an automated scan by default, you can give users a Prompt Input Bar to customize or guide the AI's behavior:

- **Automatic Scanning Mode: The user simply pastes a URL, and the system automatically audits for standard WCAG accessibility rules and visual layout bugs.**
- **Interactive Prompting Mode: The user can enter specific design or compliance instructions, such as:**
    - ***"Focus only on fixing the mobile navigation menu overlap."***
    - ***"Ensure all text elements meet strict WCAG AAA contrast guidelines for dark mode."***
    - ***"Adjust button padding and alignment without changing the brand font colors."***

2. What Does the Output (OP) Look Like?

The output is presented through a 3-Part Multi-Modal Dashboard:

Detailed Breakdown of the 3 Output Components

Part 1: The Visual Side-by-Side Preview (Image Slider)

- **What it is: An interactive UI slider in Streamlit or React.**
- **How it works: On the left side of the slider, you see the Original Broken Screenshot with bounding boxes around visual bugs. On the right side, you see the Re-Rendered Fixed Screenshot, which shows the page after Playwright injected the AI-generated CSS fix into the browser sandbox.**

Part 2: Structured Audit Report & Suggestions

- **What it is: A readable diagnostic report explaining *why* something is broken.**
- **What it contains:**
    - **Bug Classification: Categorized into *Visual Layout Defect* vs. *WCAG Accessibility Violation*.**
    - **Severity Rating: Critical, High, Medium, or Low.**
    - **Clear Explanations: e.g., *"The contrast ratio between text #777777 and background #FFFFFF is 2.8:1, failing WCAG 2.1 AA requirement of 4.5:1."***

Part 3: Surgical Code Patches (Copy-Paste / Diffs)

- **What it is: Modular, non-destructive CSS/HTML patches.**
- **Why this is critical: Re-generating an entire website from scratch destroys custom JavaScript, backend API bindings, and routing structures. VisionGuard AI outputs only the modified CSS rules or HTML snippets needed to patch the bug, allowing developers to copy and paste them directly into their existing codebase.**

Summary Matrix: What VisionGuard AI Delivers

Output Type	Included?	How It Functions
Completely New Website?	**No**	**Preserves your brand structure, code, and layout logic—it fixes bugs rather than replacing the site.**
Suggestions & Explanations?	**Yes**	**Displays a structured diagnostic audit with severity ratings and compliance rules.**
Surgical Code (CSS/HTML)?	**Yes**	**Provides clean, modular CSS/HTML patches ready to copy-paste or merge.**
Visual Preview (Before/After)?	**Yes**	**Features a side-by-side interactive visual diff slider showing real-time re-rendered fixes.**
Interactive Prompting?	**Yes**	**Allows users to give targeted instructions (e.g., *"Fix mobile contrast only"*).**

WHY OUR PROJECT IS AN AGENTIC AI PROJECT?

Yes, absolutely. The architecture of VisionGuard AI is a textbook example of Agentic AI.

To understand why—and to confidently explain it during your viva—you need to understand the difference between standard AI (like ChatGPT) and Agentic AI.

Standard generative AI is stateless and passive. You give it a prompt, it gives you an output, and it stops. It relies entirely on human supervision and a static input/output model.

Agentic AI, on the other hand, possesses agency—the capacity to act independently, purposefully, and dynamically to achieve a specific goal with minimal human intervention. It operates in a continuous loop of perceiving its environment, reasoning, calling external tools, and self-correcting.

Here is exactly how VisionGuard AI perfectly aligns with the four core pillars of Agentic AI:

1. Perception (Gathering Data Autonomously)

- **Standard AI: You have to manually take a screenshot, manually copy your CSS, and paste it into a prompt box.**
- **VisionGuard (Agentic): The AI system uses perception. It automatically navigates to the URL, spins up headless browsers via Playwright across**

multiple viewports, and extracts the DOM tree. It "looks" at its environment without you having to feed it manually.

2. Autonomous Tool Invocation (Function Calling)

- **Standard AI: Can only generate text. It cannot interact with the real world or run code.**
- **VisionGuard (Agentic): The agent does not just generate an answer; it takes action via *tool invocation*. The central LLM acts as the "brain," but it is instructed to call specific Python scripts (tools) to execute the Playwright browser, run the WCAG mathematical rules engine, and format the output.**

3. Reasoning and Planning (Chain-of-Thought)

- **Standard AI: Guesses the answer in a single, unstructured step.**
- **VisionGuard (Agentic): Employs goal-oriented reasoning. It uses frameworks like *Chain-of-Thought (CoT)* or *ReAct (Reason and Act)*. The agent breaks down the problem: *"First, I need to find the visual overlap. Next, I need to map it to the DOM. Finally, I need to rewrite the CSS."***

4. Closed-Loop Reflection (Self-Correction)

- **Standard AI: If it writes bad code, it just gives it to you, and you have to find out it doesn't work.**
- **VisionGuard (Agentic): This is the defining feature of elite agentic systems. It uses *Reflection*. After the agent writes the CSS patch, it doesn't just hand it to the user. It injects the code back into the Playwright sandbox, re-renders the page, and "looks" at the new screenshot. If the layout is still broken, it acknowledges the failure and loops back to rewrite the code until the goal is met.**

How to Say This in Your Viva

If an evaluator asks, *"Is this just a wrapper around the Gemini API?"* you will say:

"No, this is an Agentic AI system. A simple API wrapper is passive and stateless. Our system utilizes an autonomous ReAct loop. The LLM acts as the orchestrator, autonomously calling external tools like Playwright to perceive the environment, parsing the DOM, and utilizing a closed-loop sandbox to self-correct its own generated code before presenting the final result."

VisionGuard AI: Master Technical Specification & Architecture Blueprint:

Project Title: VisionGuard AI – Agentic Multimodal UI/UX & WCAG Accessibility Auditor

System Class: Autonomous Multimodal Agentic System / DevSecOps Quality Assurance Engine

Execution Timeline: 10-Week Sprint (4-Person Squad)

1. Executive Summary & Core Mission

The Problem

Modern web applications suffer from two major frontend failure points:

1. **Visual Layout Regressions: UI elements look correct on desktop browsers but suffer from overlapping text, broken flexbox alignments, or clipped buttons across mobile and tablet viewports.**
2. **Accessibility Non-Compliance: Web applications frequently violate WCAG 2.1 (Web Content Accessibility Guidelines) regulations—such as poor color contrast ratios and missing screen-reader tags—exposing organizations to legal compliance risks and excluding visually impaired users.**

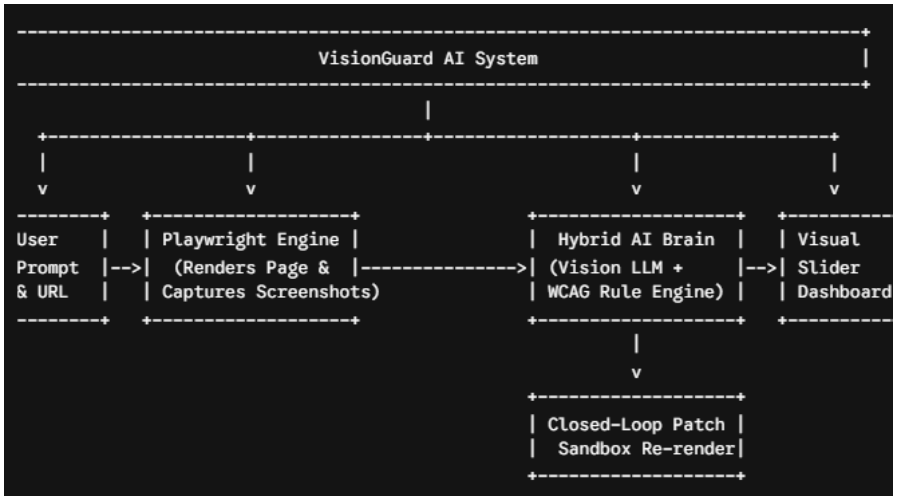
Manual quality assurance across dozens of screen sizes and accessibility checklists is slow, costly, and prone to human error.

The Solution: VisionGuard AI

VisionGuard AI is an autonomous, multimodal QA agent. Rather than acting as a static text-generation chatbot, it functions as a closed-loop automated engineer.

It autonomously renders target web pages across multiple device viewports using a headless browser, parses the underlying HTML/CSS code structure, evaluates visual and accessibility defects using a hybrid AI engine, and automatically generates and verifies surgical CSS/HTML code patches inside a live sandbox environment.

+----------



2. Deep-Dive into the AI Concepts (Your Viva Defense Engine)

To dominate your academic evaluation and technical interviews, your team must master the five foundational AI paradigms integrated into this project:

Concept 1: Multimodal Vision-Language Processing (VLM)

- **What It Is: Traditional AI processes text or images in isolation. Multimodal Vision-Language Models (such as Gemini 1.5 Pro or Claude 3.5 Sonnet) perform joint spatial and textual reasoning.**
- **How VisionGuard AI Uses It: The model receives two inputs simultaneously: a high-resolution screenshot buffer (visual matrix) and the extracted DOM/CSS code tree (textual structure). The VLM performs cross-modal alignment—mapping a visual pixel flaw (e.g., a button overlapping a text banner) directly to the specific CSS selector responsible (e.g., .nav-item { position: absolute; }).**

Concept 2: Agentic Tool Calling & ReAct Loop

- **What It Is: An Agentic AI operates in an autonomous execution loop (Reasoning + Action). Instead of outputting static text, the central LLM acts as an orchestrator that calls external Python functions ("Tools").**
- **How VisionGuard AI Uses It: The agent autonomously determines which tool to trigger based on system state:**
    - **Tool A: capture_viewport(url, device_type) -> Triggers Playwright.**
    - **Tool B: extract_dom_ast(url) -> Runs HTML/CSS parsing.**
    - **Tool C: calculate_wcag_contrast(fg_color, bg_color) -> Executes contrast formulas.**

- **Tool D: inject_css_patch(css_string) -> Re-renders the fix in the sandbox.**

Concept 3: Neuro-Symbolic AI (Hybrid Intelligence)

- **What It Is: Combining neural networks (probabilistic, flexible deep learning) with symbolic systems (deterministic, rule-based mathematical logic).**
- **How VisionGuard AI Uses It:**
  - **Symbolic Engine (Deterministic): Evaluates mathematical compliance rules with 100% precision using exact formulas. For WCAG 2.1 Color Contrast, the system calculates relative luminance (L) from linearized sRGB values:**

$$L = 0.2126R + 0.7152G + 0.0722B$$

where linearized sRGB components are calculated as:

$$R = \begin{cases} \dfrac{R_{\text{sRGB}}}{12.92}, & \text{if } R_{\text{sRGB}} \leq 0.04045 \\ \left(\dfrac{R_{\text{sRGB}} + 0.055}{1.055}\right)^{2.4}, & \text{otherwise} \end{cases}$$

The contrast ratio (C) is then evaluated deterministically:

$$C = \frac{L_1 + 0.05}{L_2 + 0.05}$$

- **Neural Engine (Probabilistic): Handles subjective visual layout assessment, detection of overlapping div elements, clipped fonts, and aesthetic layout harmony where rigid rules fail.**

Concept 4: Closed-Loop Reflection & Self-Correction

- **What It Is: An agentic pattern where the AI generates an artifact, executes it in a isolated sandbox, receives execution feedback, and iteratively refactor its code until validation passes.**
- **How VisionGuard AI Uses It:**
  1. **The AI generates a CSS patch for a broken layout.**
  2. **The backend injects this CSS patch directly into a live Playwright browser instance.**
  3. **Playwright takes a new screenshot of the modified page.**

4. **The Vision VLM compares the new screenshot against the original. If the layout flaw persists, the visual diff is fed back into the prompt context for automatic self-correction before presentation to the user.**

Concept 5: Chain-of-Thought (CoT) Code Synthesis

- **What It Is: Forcing an LLM to generate explicit intermediate logical steps before producing a final output, preventing hallucinations in code generation.**
- **How VisionGuard AI Uses It: System prompts enforce a 4-stage Chain of Thought:**
    1. ***Identify Visual Defect* -> 2. *Isolate DOM Selector* -> 3. *Diagnose Root Cause* -> 4. *Synthesize Minimal CSS Patch*.**

3. User Experience, Inputs, & Output Specification

User Inputs

1. **Target Web URL or HTML/CSS File: The public link or code snippet to audit.**
2. **Optional Custom Prompt: Natural language guidance from the developer (e.g., *"Focus specifically on fixing dark-mode contrast issues on mobile screens"* or *"Ensure the navigation menu remains fixed without changing header font size"*).**

System Outputs (The 3-Part Multi-Modal Dashboard)

4. 4-Person Team Roles & Tech Stack Assignment

To prevent team friction and guarantee parallel execution, every team member owns a specific microservice boundary:

| Role | Primary Responsibility | Assigned Tech Stack | Key Deliverables |

| :--- | :--- | :--- | :--- |

| **Role 1: Lead AI Agent Architect** | System prompts, VLM orchestration, tool-calling chains, self-correction reflection loop. | LangChain / LangGraph, Gemini Vision API / Claude 3.5 Sonnet, Python | `agent_engine.py`, `prompt_templates.py`, `reflection_loop.py` |

| **Role 2: Headless Automation Engineer** | Headless browser execution, multi-viewport rendering, screenshot buffers, CSS sandbox injection. | Playwright, Python Asyncio, Docker | `browser_engine.py`, `viewport_manager.py`, `sandbox_injector.py` |

| **Role 3: DOM & Accessibility Engineer** | HTML/CSS parsing, DOM tree abstraction, deterministic WCAG 2.1 math engine. | BeautifulSoup4, TinyCSS2, Python NumPy | `dom_parser.py`, `wcag_checker.py`, `ast_extractor.py` |

| **Role 4: Backend Microservices & UI Lead** | Asynchronous API gateway, Redis caching layer, interactive Streamlit slider UI. | FastAPI, Redis, Streamlit, Streamlit-Image-Comparison | `main_api.py`, `app_dashboard.py`, `redis_cache.py` |

---

5. The 10-Week Master Agile Roadmap

This roadmap breaks down every single week into actionable, role-specific engineering tasks.

Phase 1: Environment, Automation, & Parsing (Weeks 1–3)

Week 1: Repository Initialization & Workspace Architecture

*** **All Roles:** Initialize GitHub repository with standard microservices directory structure. Configure Docker containers and virtual environments.**

*** **Role 1:** Obtain API keys for Gemini Vision / Claude 3.5 and write basic API connectivity tests.**

*** **Role 2:** Install Playwright, download Chromium binaries, and configure basic headless browser invocation.**

*** **Role 3:** Set up HTML parsing libraries (`BeautifulSoup4`, `TinyCSS2`) and build sample test HTML/CSS input files.**

*** **Role 4:** Initialize FastAPI application skeleton with basic `/health` endpoints and CORS middleware.**

Week 2: Multi-Viewport Headless Capture Pipeline

*** **Role 1:** Define the initial JSON schema for outputting visual bug diagnostic reports.**

*** **Role 2:** Write `browser_engine.py` using Playwright to take a target URL and asynchronously capture full-page PNG screenshot buffers at standard breakpoints:**

*** Mobile (`375x667`)**

*** Tablet (`768x1024`)**

*** Desktop (`1920x1080`)**

*** **Role 3:** Build script to strip inline CSS and external stylesheet rules into an aggregated CSS object.**

*** **Role 4:** Build Streamlit UI wireframe allowing URL input submission and basic image rendering.**

Week 3: Deterministic WCAG Accessibility Engine

*** **Role 1:** Draft initial system prompts instructing the VLM on how to read UI bounding boxes.**

*** **Role 2:** Implement cookie-banner bypass and pop-up dismissal logic inside Playwright scripts to ensure clean captures.**

*** **Role 3:** Implement `wcag_checker.py`. Write exact RGB-to-relative-luminance and contrast-ratio calculation functions ($C = \frac{L_1 + 0.05}{L_2 + 0.05}$). Program checks for missing `alt` attributes and missing `aria-label` tags.**

*** **Role 4:** Connect FastAPI `/audit/static` route to receive URLs and return deterministic WCAG violation JSON data.**

Phase 2: Multimodal AI Integration & Patching (Weeks 4–6)

Week 4: Visual Inspection VLM Integration

*** **Role 1:** Connect Gemini Vision / Claude 3.5 Sonnet to process screenshot buffers alongside raw DOM code. Instruct model to identify visual overlap, font clipping, and layout misalignment.**

*** **Role 2:** Enhance Playwright scripts to capture bounding box coordinates (`x, y, width, height`) of suspected broken elements.**

*** **Role 3:** Refine DOM parser to isolate specific CSS selector blocks associated with elements flagged by the visual engine.**

*** **Role 4:** Update FastAPI backend to handle multi-part data payload routing (Images + JSON DOM trees).**

Week 5: Chain-of-Thought Patch Generation Engine

*** **Role 1:** Build `patch_generator.py`. Implement Chain-of-Thought prompting to produce modular, non-destructive CSS overrides wrapped strictly in JSON blocks.**

*** **Role 2:** Build `sandbox_injector.py`. Create a Playwright function that takes a raw CSS string, injects it into a live page DOM (`page.add_style_tag`), and captures a "Fixed" screenshot.**

*** **Role 3:** Build validator ensuring generated CSS patches do not contain syntax errors or invalid properties.**

*** **Role 4:** Integrate Redis caching. Store previously audited URLs and generated patches to reduce API costs and guarantee sub-3-second responses for cached queries.**

Week 6: System Microservices Integration

*** **Role 1:** Connect the AI Agent orchestrator with the DOM parser and Playwright sandbox module.**

*** **Role 2:** Optimize Playwright performance using `wait_for_load_state('domcontentloaded')` to reduce browser capture latency.**

*** **Role 3:** Combine deterministic WCAG violation outputs with probabilistic VLM visual reports into a unified diagnostic JSON object.**

*** **Role 4:** Connect FastAPI backend seamlessly to Streamlit frontend using background task workers (`Celery` or FastAPI `BackgroundTasks`).**

Phase 3: Closed-Loop Validation & UI Polish (Weeks 7–9)

Week 7: Closed-Loop Reflection Sandbox (Self-Correction)

*** **Role 1:** Implement the autonomous reflection loop. If the re-rendered sandbox screenshot still contains layout flaws, feed the visual diff back to the VLM to iteratively refine the CSS patch.**

*** **Role 2:** Fine-tune image-cropping scripts to crop screenshots around specific failing UI components before sending them to the VLM, cutting token overhead by up to 80%.**

*** **Role 3:** Add support for detecting light-mode vs. dark-mode CSS variables (`prefers-color-scheme`).**

*** **Role 4:** Integrate `streamlit-image-comparison` component into the UI dashboard for interactive side-by-side visual diff sliding.**

Week 8: Dashboard Customization & Custom Prompting

*** **Role 1:** Implement support for custom user guidance prompts (e.g., *"Focus on mobile button contrast"*).**

*** **Role 2:** Containerize the entire execution pipeline (FastAPI + Playwright + Headless Chromium) inside Docker.**

*** **Role 3:** Add export functions (PDF/JSON) allowing users to download generated audit reports and CSS patch files.**

*** **Role 4:** Polish UI styling, adding severity color badges (Red = Critical, Yellow = Medium, Green = Compliant) and copy-to-clipboard code blocks.**

Week 9: End-to-End Stress Testing & Latency Optimization

*** **All Roles:** Conduct end-to-end stress tests across 30+ real-world public websites (university portals, e-commerce templates, news sites).**

*** **Role 1 & 3:** Tune prompt tokens and JSON parsing logic to eliminate edge-case LLM formatting errors.**

*** **Role 2 & 4:** Optimize Redis caching and async routing to guarantee responsive dashboard updates.**

Phase 4: Freeze & Presentation Preparation (Week 10)

Week 10: Codebase Freeze & Viva Defense Preparation

*** **All Roles:** **FREEZE CODEBASE.** Do not add any new features.**

*** **Role 1 & 4:** Prepare live presentation flow using pre-verified public website URLs. Set up local fallback responses in case of internet instability during the demo.**

*** **Role 2 & 3:** Draft the formal IEEE-format project report, complete with system block diagrams, sequence flows, and WCAG mathematical formulas.**

6. Viva Defense Shield: Crushing Evaluator Objections

Prepare your team with these exact answers for tough evaluator questions:

Question 1: "Why use AI for accessibility when traditional automated tools like Lighthouse already exist?"

*** **Your Answer:** *"Traditional tools like Lighthouse rely 100% on static DOM rules. They can check if an `alt` tag exists, but they cannot evaluate visual semantics. Lighthouse cannot tell if a mobile navigation menu visually overlaps a hero banner, or if custom canvas elements are visually illegible. VisionGuard AI combines deterministic rule-checking for WCAG formulas with Multimodal Vision AI to detect visual layout regressions that static code checkers miss entirely."***

Question 2: "What prevents the AI from generating bad CSS code that breaks the rest of the website?"

*** **Your Answer:** *"We use a Closed-Loop Neuro-Symbolic Sandbox. The AI is restricted to outputting scoped, modular CSS patches. Before any patch is shown to the user, our backend injects the CSS into a headless Playwright instance, re-renders the page, and executes a reflection check. If the generated fix breaks surrounding layout components, the system catches the error in the sandbox and self-corrects automatically."***

Question 3: "Isn't this system slow and expensive due to sending full screenshots to Vision APIs?"

*** **Your Answer:** *"We engineered two latency and cost optimization pipelines. First, our DOM parser crops screenshots down to specific bounding box coordinates around the failing element before API transmission, cutting token consumption by up to 80%. Second, we implemented an asynchronous Redis caching layer so repeated scans on a domain respond in milliseconds."***

Question 4: "Is there any data privacy or PII compliance risk with this project?"

*** **Your Answer:** *"Zero. VisionGuard AI operates exclusively on publicly accessible web URLs and rendered client-side DOM/CSS trees. It does not ingest internal server logs, private database records, or sensitive user information, making it 100% compliant with GDPR and data privacy standards."***

Final Visual Appearance & User Flow of VisionGuard AI:

FINAL DASHBOARD LAYOUT:

Final Visual Appearance & User Flow of VisionGuard AI

When you launch VisionGuard AI on evaluation day, your dashboard will look like an enterprise-grade developer tool, completely distinct from standard student chatbots.

The Final Dashboard Layout

Step-by-Step User Flow During Live Demo

1. **URL Input:** You type or paste a public web link into the input bar.

2. **Execution Telemetry:** As the backend works, a status log displays live terminal updates, showing that Playwright, the DOM parser, the deterministic contrast engine, and the multimodal LLM are executing in sequence.

3. **Interactive Comparison Slider:** The main screen displays an interactive slider component (`streamlit-image-comparison`). Dragging the slider shows the original site with red error bounding boxes alongside the AI-corrected page, re-rendered in real time by Playwright.

4. **Diagnostic Matrix:** A structured table categorizes every detected bug by severity, element selector, and WCAG specification number.

5. **Code Output Box:** A syntax-highlighted code block contains the clean, modular CSS patch with a **Copy Code** button.

The Master Matrix of Viva "Bouncers" & Complete Answers

This section covers technical questions, corner cases, and skeptical queries evaluators or professors might ask during your presentation.

Category A: Core AI & Agentic Mechanics

Q1: "Isn't this project just a thin wrapper around the Gemini or Claude Vision API?"

*** **Evaluator Intent:** Testing if you wrote actual system architecture or just copied 10 lines of API integration code.**

*** **Short Answer:** "No. An API wrapper is passive and stateless—it accepts text and outputs text. VisionGuard AI is an autonomous, closed-loop agentic state machine. The LLM handles vision processing, while our system manages asynchronous browser automation, deterministic mathematical WCAG verification, Abstract Syntax Tree parsing, and a closed-loop sandbox that tests and self-corrects generated code before displaying it."**

*** **Technical Deep-Dive:****

*** Explain the **ReAct (Reason + Act)** loop: The LLM does not just generate an answer; it calls external tools using JSON function schemas (`capture_viewport`, `extract_dom_ast`, `re_render_patch`).**

*** Emphasize **Neuro-Symbolic Architecture**: The AI (Neural) handles subjective visual bug detection, while a Python engine (Symbolic) calculates exact color contrast ratios using sRGB formulas with 100% mathematical precision.**

Q2: "Why use a Large Multimodal Model (VLM) instead of classic Computer Vision like YOLO or OpenCV?"

*** **Evaluator Intent:** Checking if you understand why deep learning / generative models are required instead of standard image processing.**

*** **Short Answer:** "Object detection models like YOLO or OpenCV can locate pixel boundaries, but they lack semantic understanding of web code. They cannot look at a cropped image, infer that a button suffers from a flexbox layout collapse, and output the exact CSS selector needed to fix it. VLMs perform joint reasoning across visual pixels and HTML/CSS text structures simultaneously."**

*** **Technical Deep-Dive:****

*** OpenCV can identify overlapping contours, but it cannot map an image contour to an Abstract Syntax Tree node (`<div class="nav">`).**

*** VLMs possess **cross-modal alignment**: they map visual anomalies directly to structural code concepts.**

Q3: "How do you prevent the LLM from hallucinating invalid CSS syntax or breaking page layouts?"

* **Evaluator Intent:** Probing your system's error-handling and code validation guardrails.

* **Short Answer:** "Through our **Closed-Loop Sandbox Reflection Loop**. We never show raw AI output directly to the user. Generated CSS patches are parsed through a CSS AST validator (`TinyCSS2`) to check for syntax errors, injected into a headless Playwright browser instance, and re-scanned. If the layout breaks or syntax fails, the error is fed back to the agent for automatic self-correction."

* **Technical Deep-Dive:**

  * Show the 4-step Chain-of-Thought (CoT) prompting strategy:

    1. *Identify Visual Defect*

    2. *Isolate DOM Selector*

    3. *Diagnose Root Cause*

    4. *Synthesize Minimal Scoped Patch*

Category B: Accessibility Standards & Mathematics

Q4: "Explain the exact mathematical formula for WCAG 2.1 color contrast calculation."

* **Evaluator Intent:** Testing your theoretical knowledge of image processing and accessibility rules.

* **Short Answer:** "WCAG 2.1 contrast evaluation measures the relative luminance (L) of foreground and background colors. Relative luminance measures the perceived brightness of a color normalized between 0 for darkest black and 1 for lightest white."

* **Technical Deep-Dive:**

  * **Step 1: Linearize sRGB values.** For each 8-bit color channel ($R_{\text{sRGB}}, G_{\text{sRGB}}, B_{\text{sRGB}} \in [0, 255]$), compute the normalized channel value $c = C / 255$:

$$c_{\text{linear}} = \begin{cases} \frac{c}{12.92}, & \text{if } c \le 0.04045 \\ \left(\frac{c + 0.055}{1.055}\right)^{2.4}, & \text{otherwise} \end{cases}$$

* **Step 2: Calculate Relative Luminance (L).** Using the CIE colorimetric coefficients:

$$L = 0.2126 R_{\text{linear}} + 0.7152 G_{\text{linear}} + 0.0722 B_{\text{linear}}$$

* **Step 3: Calculate Contrast Ratio (C).** Comparing the lighter luminance (L_1) and darker luminance (L_2):

$$C = \frac{L_1 + 0.05}{L_2 + 0.05}$$

* **Compliance Thresholds:**

  * WCAG 2.1 Level AA (Normal Text): $C \ge 4.5:1$

  * WCAG 2.1 Level AA (Large Text $\ge 18\text{pt}$ or 14pt bold): $C \ge 3.0:1$

  * WCAG 2.1 Level AAA (Normal Text): $C \ge 7.0:1$

Q5: "Why can't static analysis engines like Axe-core or Google Lighthouse solve all these problems?"

* **Evaluator Intent:** Checking if you know the limitations of existing commercial tools.

* **Short Answer:** "Static analyzers like Axe-core inspect the DOM tree without evaluating visual rendering context. They miss visual layout regressions—such as elements overlapping due to `z-index` conflicts, absolute positioning glitches, or dynamic CSS flexbox collapses—because those defects exist only in the rendered visual frame, not the raw HTML text."

Category C: Automation & Web Architecture

Q6: "Why did you choose Playwright over Selenium or Puppeteer?"

* **Evaluator Intent:** Testing your technical stack justification.

* **Short Answer:** "Playwright provides native asynchronous support in Python, runs faster than Selenium by using direct browser debugging protocols rather than HTTP web drivers, supports multi-context execution (rendering Mobile, Tablet, and Desktop viewports in parallel), and handles dynamic DOM updates natively."

* **Technical Comparison Table:**

| Feature | Selenium | Puppeteer | Playwright (Chosen) |

| :--- | :--- | :--- | :--- |

| **Architecture** | HTTP WebDriver (Slower) | Chrome DevTools Protocol | Chrome / Firefox / WebKit Protocols |

| **Async Support** | Limited / Blocking | Node.js Native | Python `asyncio` Native |

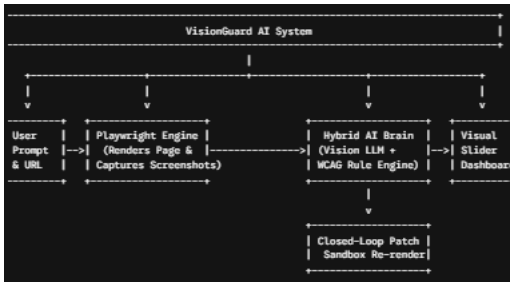
| **Multi-Viewport Contexts** | High Memory Overhead | Chromium Only | Extremely Lightweight Contexts |

| **Auto-Waiting** | Requires Manual Sleep Statements | Basic | Automatic Element State Checks |

Q7: "What if a target website blocks headless browsers using Cloudflare or CAPTCHA?"

* **Evaluator Intent:** Challenging your system's resilience against real-world web barriers.

* **Short Answer:** "Our primary deployment target is internal development pipelines (CI/CD environments, local staging servers, and open-source design repositories) where bot protection is disabled. However, for public URLs, our Playwright engine uses custom user-agent spoofing, realistic viewport flags, and `stealth` context configurations to bypass basic automated bot detection."



Category D: Backend Performance & Optimization

Q8: "Sending high-resolution full-page screenshots to Vision APIs is slow and expensive. How do you manage latency and token costs?"

* **Evaluator Intent:** Testing your awareness of cost efficiency, latency, and scalability.

* **Short Answer:** "We use a **Bounding Box Spatial Cropping Pipeline**. Instead of sending an entire 1920×1080 pixel screenshot to the Vision API, our DOM

parser flags candidate elements with potential errors, crops tight image bounding boxes around those specific regions, and sends only those small image segments. This reduces API token consumption and processing latency by up to 80%."

Q9: "How does FastAPI handle high concurrency when multiple users request website audits simultaneously?"

*** **Evaluator Intent:** Assessing your backend microservices architecture knowledge.**

*** **Short Answer:** "FastAPI uses Python's asynchronous event loop (`async/await`). Non-blocking I/O operations—such as fetching web pages, making LLM API calls, and reading from the database—yield control back to the event loop. Heavy browser rendering tasks are offloaded to asynchronous background worker queues using Redis and Celery, preventing main thread blocking."**

Final Pre-Viva Technical Checklist

Before walking into your presentation:

1. Ensure your local virtual environment or Docker container has Chromium binaries pre-installed (`playwright install chromium`).

2. Have pre-cached demonstration results stored in Redis for your target test URLs, ensuring your demo works smoothly even if the internet connection fluctuates.

3. Keep the WCAG contrast formulas written down in your presentation slide deck to showcase mathematical rigor.
